



Bangladesh University of Business and Technology

Course Title: Computer Networks Lab

Course Code: CSE 320

ASSIGNMENT

Experiment Name: HTTP Client-Server Configuration

Experiment No.: 01

Submitted By:

Name:	Shadman Shahriar
ID:	20245103408
Intake:	53
Section:	01

Submitted To:

Course Teacher:	Dr. Khandoker Nadim Parvez
Designation:	Associate Professor
Department:	Computer Science and Engineering

Submission Date: 13 April 2026

Introduction

Hypertext Transfer Protocol (**HTTP**) is the application-layer protocol that web browsers and web servers use to communicate with each other over the Internet. It is the foundation of any data exchange on the web. HTTP Client-Server describes a standardized communication model where the client initiates a request and the server provides a response. It is a reliable communication over **TCP**.

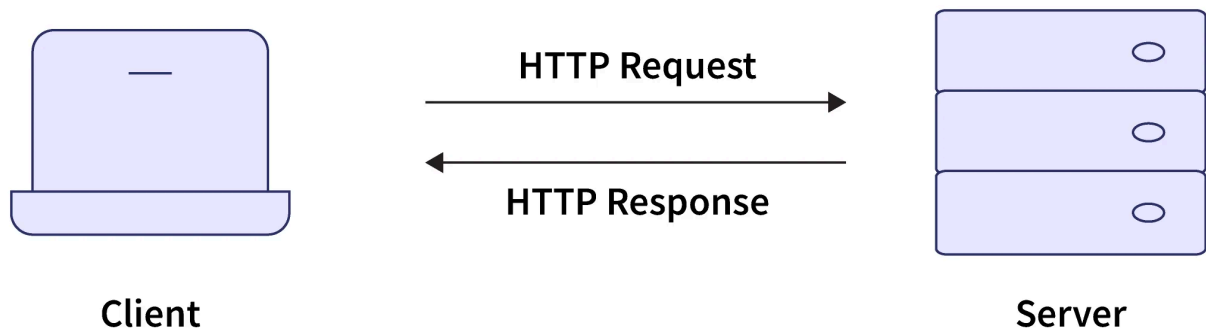


Figure 1.1: Client-Server communication over HTTP.

A basic HTTP Client-Server configuration consists of three core components:

1. **Client:** A piece of software program, typically a web browser that initiates communication by requesting specific resources.
2. **The Network:** The medium (**Internet** or **Intranet**) using **TCP/IP protocols** to transport data between endpoints.
3. **Server:** A computer or software (e.g., Nginx, Apache, Socket) that listens for requests, processes logic, and serves back the requested data.

Structure of HTTP request and response:

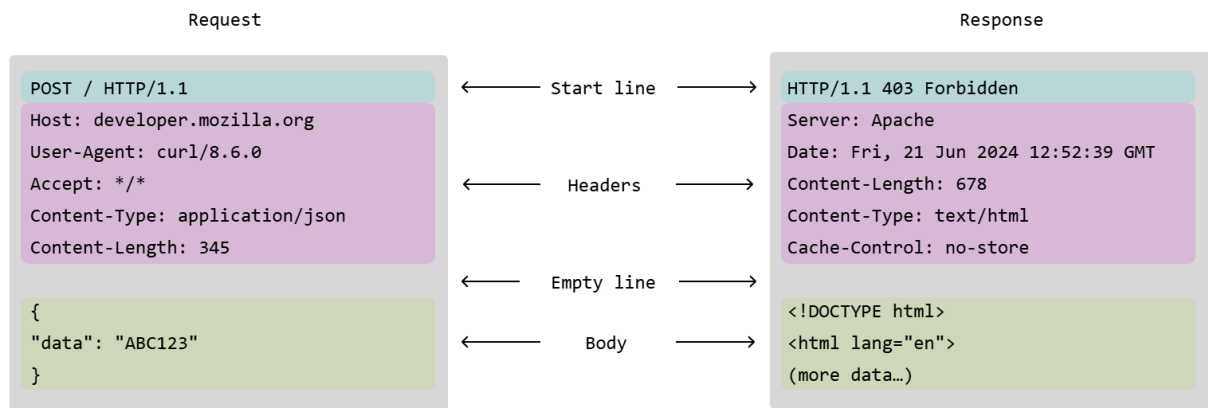


Figure 1.2: HTTP request and response structure.

HTTP Request Structure

An HTTP request consists of the following parts,

- **Request Line:** The first line, which includes:
 - **Method:** A verb like GET, POST, PUT, or DELETE.
 - **Request-URI:** The path or endpoint of the resource (e.g., /api/users).
 - **HTTP Version:** The protocol version, typically **HTTP/1.1**.
- **Request Headers:** Case-insensitive key-value pairs (e.g., Host: localhost) providing metadata like authentication, preferred language, or content type.
- **Empty Line:** A mandatory blank line indicating the end of the headers.
- **Body (Optional):** Contains data sent to the server, commonly used with POST, PUT, or PATCH methods (e.g., JSON payload).

HTTP Response Structure

An HTTP response consists of the following parts,

- **Status Line:** The first line, which includes:
 - **HTTP Version:** The protocol version, typically **HTTP/1.1**.
 - **Status Code:** A 3-digit number indicating success (200) or failure (404).
 - **Reason Phrase:** A brief human-readable description of the status code.
- **Response Headers:** Metadata about the response or server environment (e.g., Content-Type, Set-Cookie, Server).
- **Empty Line:** Separates the metadata from the payload.
- **Body:** The actual content being returned, such as an HTML document, JSON data, or an image.

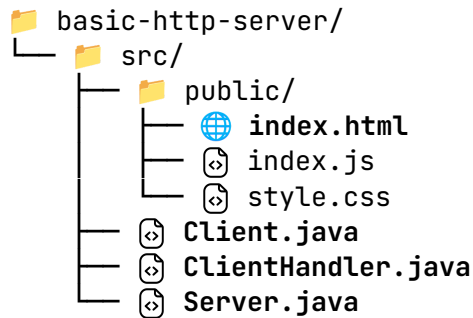
A **network socket** is an internal endpoint for sending or receiving data. It is identified by an IP address and a Port number. While HTTP manages the format and semantic meaning of messages (**requests and responses**), sockets provide the actual data pipe through which these messages travel.

In this experiment, we will implement a basic HTTP Client-Server configuration using sockets. We will use **Java** as the programming language of our choice and its built-in packages and use the network APIs.

In a real-world scenario, a client and a server can exist on different networks. But for simplicity, we will test our implementation on localhost.

Procedure

To implement our basic HTTP client-server, we create a new Java project with the following project structure:



The code is mainly divided into two parts -

1. Client Side (*Client.java*)
2. Server Side (*Server.java*, *ClientHandler.java*)

When a client sends a request to the server, the server response with an index.html that is located in the **public/** directory.

Client.java

```
/**
 * ≡≡≡ Client.java ≡≡≡
 * @date 12-Apr-2026
 * @author S. Shahriar <20245103408@cse.bubt.edu.bd>
 */

import java.io.*;
import java.net.*;

public class Client {
    // the endpoint from which the network requests
    // would be served.
    static final int port = 5500;

    // flash data buffer automatically.
    static final boolean autoFlash = true;
```

```

public static void main(String[] args) {
    try {
        // creates a stream socket and connect
        // it to the specified port number on
        // the named host.
        Socket socket = new Socket("localhost", port);

        // set up a writer instance that
        // can send requests to the server.
        OutputStream os = socket.getOutputStream();
        PrintWriter out = new PrintWriter(os, autoFlush);

        // set up a reader instance for reading
        // incoming responses from the server.
        InputStream is = socket.getInputStream();
        InputStreamReader isr = new InputStreamReader(is);
        BufferedReader in = new BufferedReader(isr);

        out.println("GET / HTTP/1.1\r\nHost: localhost\r\n");

        // read the server response line by line.
        String response;
        while ((response = in.readLine()) != null)
            System.out.println(response);

        // close the connection gracefully.
        socket.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

Server.java

```
/**
 * ≡≡≡ Server.java ≡≡≡
 * @date 12-Apr-2026
 * @author S. Shahriar <20245103408@cse.bubt.edu.bd>
 */

import java.io.*;
import java.net.*;
import java.util.concurrent.*;

public class Server {
    // the endpoint from which the network requests
    // would be served.
    static final int port = 5500;

    // the server can serve up to 10 clients at any point.
    static final int maxConcurrentClients = 10;

    public static void main(String[] args) {
        try {
            // create a new instance of a server socket
            // that waits for requests to come in over
            // the network.
            try (ServerSocket serverSocket = new ServerSocket(port)) {
                System.out.println("❗ Basic HTTP Server");
                System.out.println("✅ Socket ready on port " + port);
                System.out.println("✅ Open this link in the browser:
http://localhost:" + port);

                // using a thread pool ensures that at any point,
                // at most n threads will be active processing tasks.
                ExecutorService pool =
                    Executors.newFixedThreadPool(maxConcurrentClients);

                // run the server forever using an eternal loop
                for (;;) {
                    // accept an incoming connection to the socket.
                    Socket socket = serverSocket.accept();
```

```
        // create a new Runnable instance of ClientHandler.
        ClientHandler ch = new ClientHandler(socket);

        // create a new Thread from the Runnable instance.
        Thread clientThread = new Thread(ch);

        // add the Thread to the pool and execute it.
        pool.execute(clientThread);
    }
}
} catch (IOException e) {
    e.printStackTrace();
}
}
}
```

ClientHandler.java

```
/**
 * ≡≡≡ ClientHandler.java ≡≡≡
 * @date 12-Apr-2026
 * @author S. Shahriar <20245103408@cse.bubt.edu.bd>
 */

import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;

public class ClientHandler implements Runnable {
    private Socket socket;

    // flush data buffer automatically.
    private boolean autoFlash = true;

    // serve static files from this directory.
    private String rootDir = "src/public";

    public ClientHandler(Socket socket) {
        this.socket = socket;
    }

    // assign the appropriate MIME types
    // based on file extension.
    private String getContentType(String fpath) {
        String path = fpath.toLowerCase();
        if (path.endsWith(".html")) return "text/html; charset=UTF-8";
        if (path.endsWith(".css")) return "text/css";
        if (path.endsWith(".js")) return "application/javascript";
        if (path.endsWith(".png")) return "image/png";
        if (path.endsWith(".jpg")) return "image/jpeg";
        return "text/plain";
    }
}
```

```

// shows remote IP address and port number
// to which the socket is connected.
void printClientInfo(Socket socket) throws IOException {
    System.out.println(
        "\n✅ Client connected on "
        + socket.getInetAddress()
        + ":"
        + socket.getPort());
}

// close the socket connection.
void closeClient(Socket socket) throws IOException {
    socket.close();
    System.out.println(
        " — ❌ Connection closed on "
        + socket.getInetAddress()
        + ":"
        + socket.getPort()
    );
}

void serveFile(String request, PrintWriter out) throws IOException {
    if (request == null) return;

    String[] parts = request.split(" "); // e.g.: "GET /style.css HTTP/1.1"
    String path = parts[1]; // e.g.: "/style"

    // set the homepage path.
    if (path.equals("/")) path = "/index.html";

    // create a File object and
    // check whether the requested
    // file exists in the disk.
    File file = new File(rootDir + path);
    if (file.exists() && !file.isDirectory()) {
        // read file data as a byte array.
        byte[] fileData = Files.readAllBytes(file.toPath());

        // set appropriate MIME type for the requested file.
        String contentType = getContentType(path);
    }
}

```

```

        // construct the response body.
        String body = "HTTP/1.1 200 OK\r\n"
            + "Content-Type: " + contentType + "\r\n\r\n"
            + new String(fileData, StandardCharsets.UTF_8);

        // send the response to the client.
        out.println(body);
    }

    // if the requested file was not found:
    // construct a basic 404 response and
    // sent it to the client.
    else {
        String notFound = "<!DOCTYPE html><body><h1>
            404 Not Found</h1></body></html>";

        String body = "HTTP/1.1 404 Not Found\r\n"
            + "Content-Type: text/html; charset=UTF-8\r\n"
            + "Content-Length: " + notFound.length() + "\r\n\r\n"
            + notFound;

        out.println(body);
    }
}

@Override
public void run() {
    try {
        printClientInfo(socket);

        // set up a reader instance for reading
        // incoming requests from the client.
        InputStream is = socket.getInputStream();
        InputStreamReader isr = new InputStreamReader(is);
        BufferedReader in = new BufferedReader(isr);

        // set up a writer instance that
        // can send responses to the client.
        OutputStream os = socket.getOutputStream();
        PrintWriter out = new PrintWriter(os, autoFlush);
    }
}

```

```

// flag for determining whether the
// requested file was served.
boolean servedFile = false;

// read the client request line by line.
String line;
while ((line = in.readLine()) != null && !line.isEmpty()) {
    System.out.println(" — " + line);

    // serve the file once.
    if (!servedFile) {
        serveFile(line, out);
        servedFile = true;
    }
}
} catch (Exception e) {
    e.printStackTrace();
} finally {
    try {
        // close the connection once
        // all the requests has been
        // fulfilled.
        closeClient(socket);
    }
    catch (IOException e) {
        e.printStackTrace();
    }
}
}
}
}

```

The 📁 **public/** directory consists of basic HTML, CSS, and JavaScript files that the server will serve upon client's request.

index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8"/>
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
    <title>Random Cats</title>
    <link rel="stylesheet" href="/style.css"/>
    <script src="/index.js" defer></script>
  </head>
  <body>
    <main>
      
    </main>
  </body>
</html>
```

style.css

```
* {  
  padding: 0;  
  margin: 0;  
  box-sizing: border-box;  
}  
  
html,  
body {  
  height: 100%;  
  overflow: hidden;  
  user-select: none;  
}  
  
body {  
  background-color: #211e20;  
  display: grid;  
  place-items: center;  
}  
  
main {  
  max-width: 700px;  
  margin: 1em 1.5em;  
}  
  
img {  
  max-width: 100%;  
  max-height: 600px;  
  border-radius: 10px;  
  box-shadow: 0 0.7em 3.2em rgba(0, 0, 0, 0.3);  
  transform: rotate(-3deg) scale(100%);  
  transition: all ease 200ms;  
}  
  
img:active {  
  transform: rotate(-2deg) scale(90%);  
}
```

index.js

```
function getRandomCat() {  
  const img = document.getElementsByTagName('img')[0]  
  img.style.opacity = '0'  
  
  fetch('https://api.thecatapi.com/v1/images/search')  
    .then(function (response) {  
      response.json().then(function (res) {  
        img.src = res[0].url + '?' + Date.now()  
      })  
    })  
  
  img.onload = function () {  
    img.style.opacity = '1'  
  }  
}
```

Output

After following all the necessary steps above, we can run the **Server.java** program by pressing **Shift + F5** or clicking the **Run** icon:

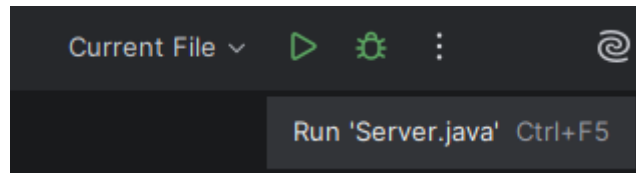


Figure 2.1: Running the server script.

Once the code gets compiled, we should see the following output in the integrated terminal:

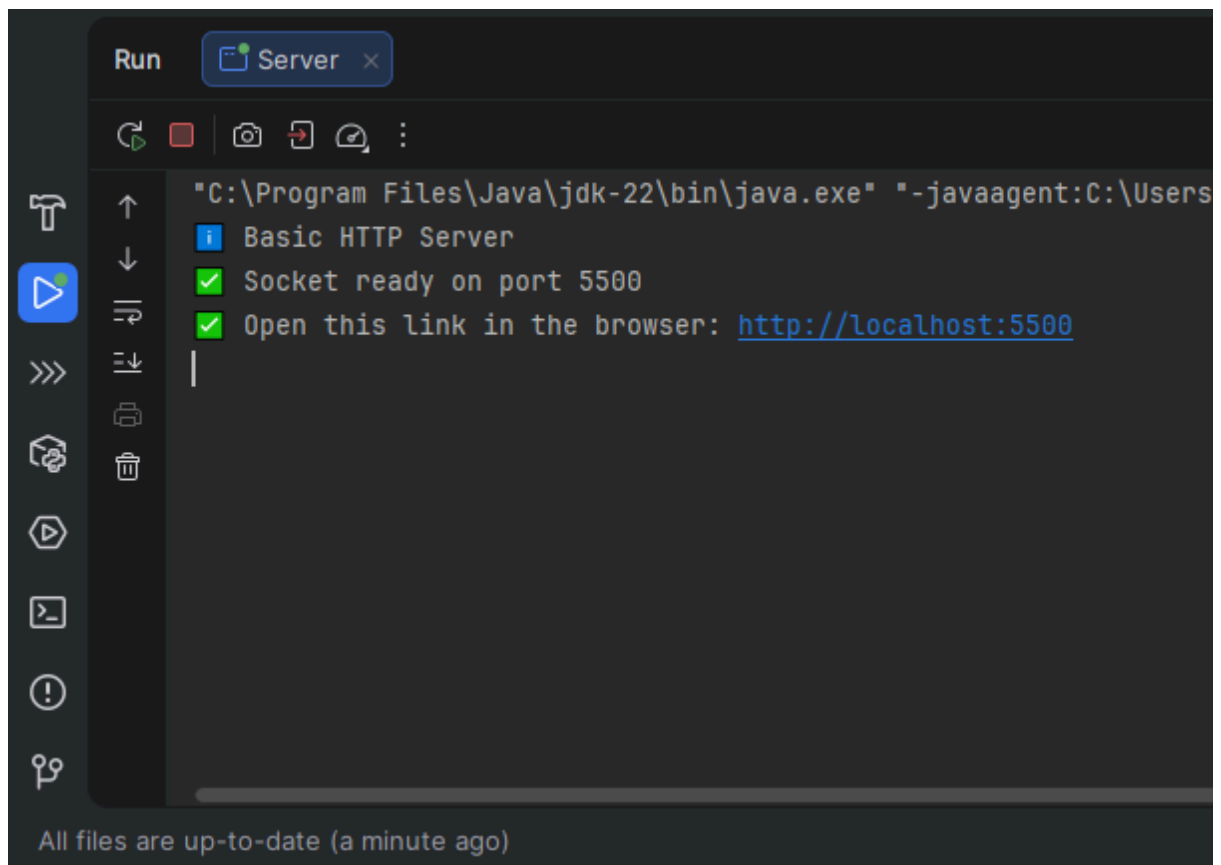
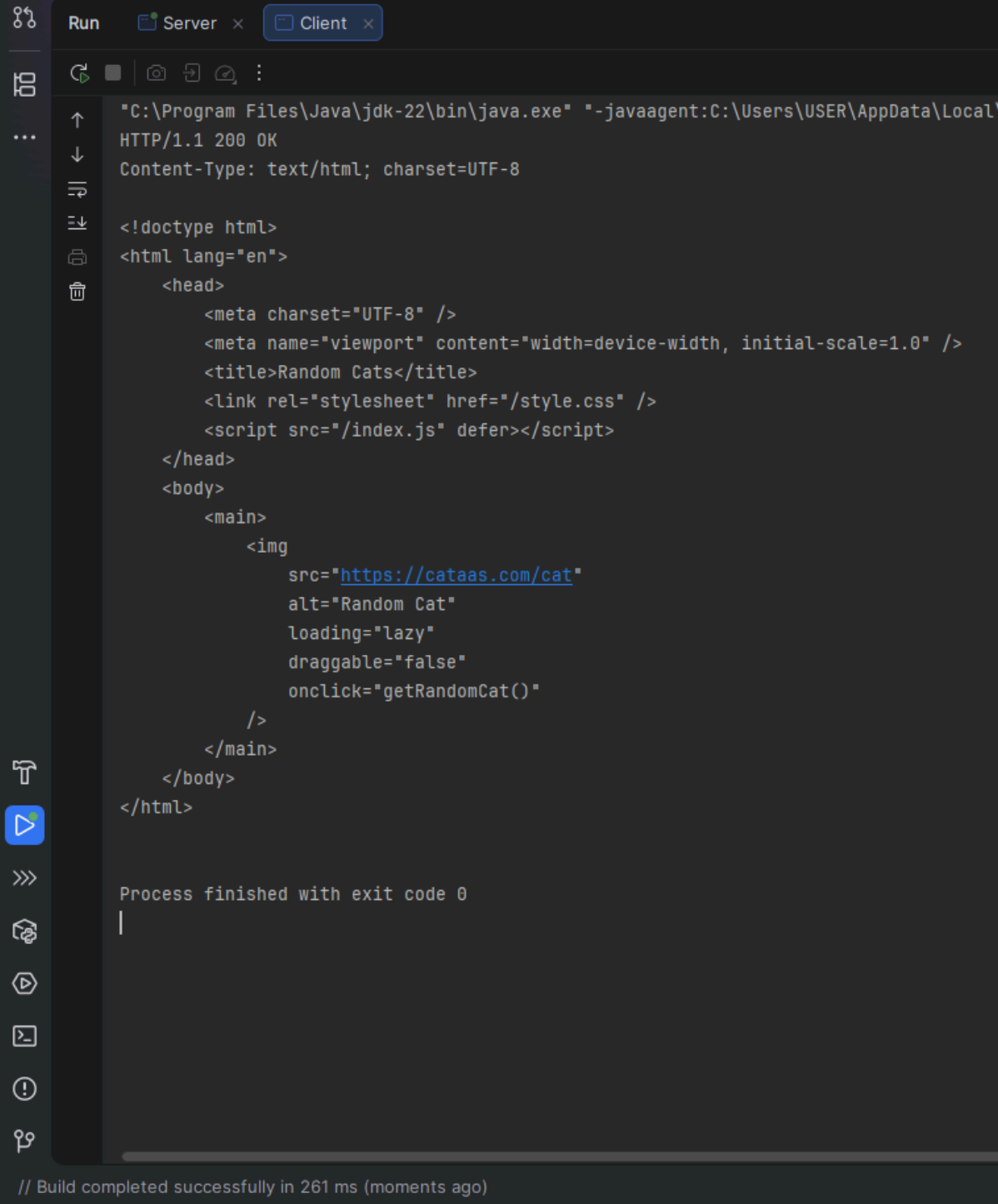


Figure 2.2: Initial output of the server side script.

Now we run the **Client.java** program.



```
"C:\Program Files\Java\jdk-22\bin\java.exe" "-javaagent:C:\Users\USER\AppData\Local\
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

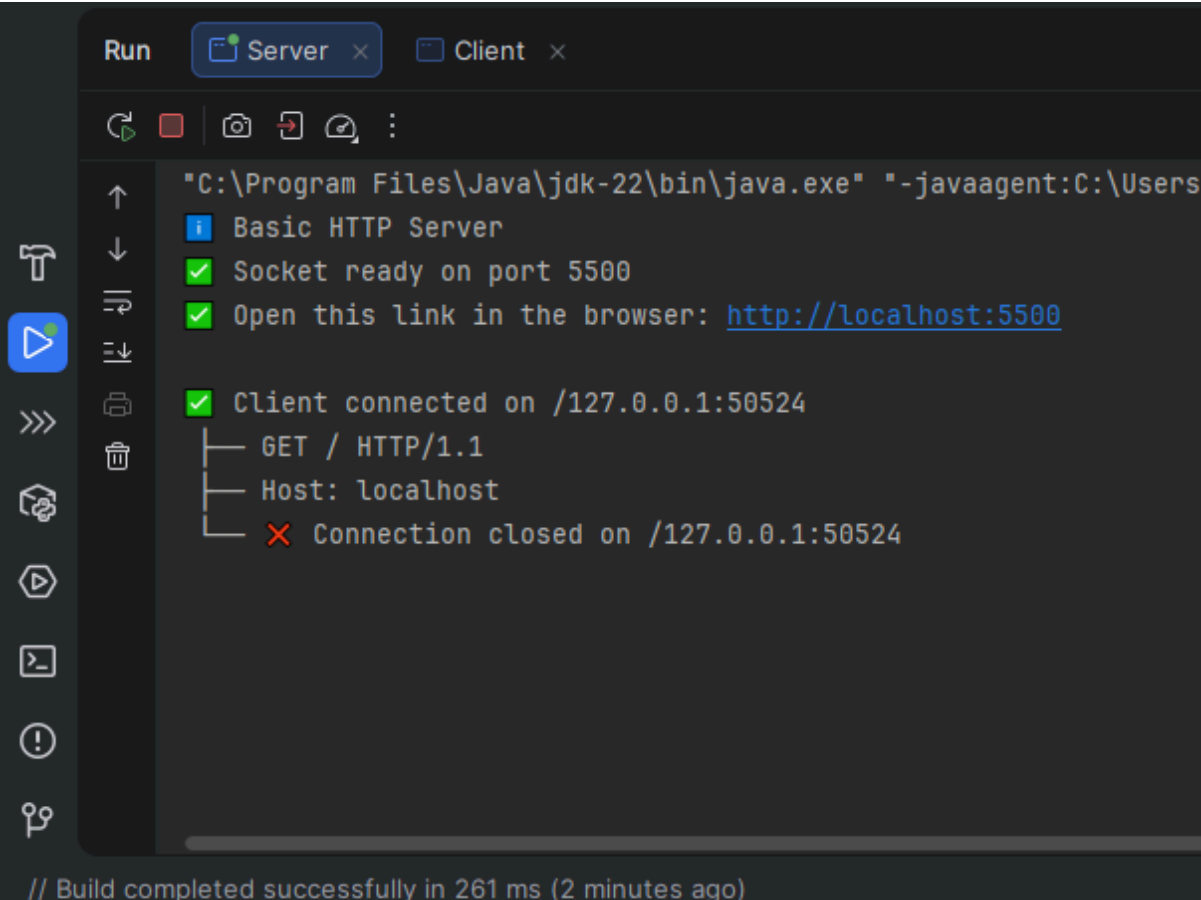
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Random Cats</title>
    <link rel="stylesheet" href="/style.css" />
    <script src="/index.js" defer></script>
  </head>
  <body>
    <main>
      
    </main>
  </body>
</html>

Process finished with exit code 0
|

// Build completed successfully in 261 ms (moments ago)
```

Figure 2.3: Output of the client side script.

Having done that, we can see the request information that was made by the client to the server:



The screenshot shows an IDE's Run console with two tabs: 'Server' and 'Client'. The 'Server' tab is active, displaying the following output:

```
"C:\Program Files\Java\jdk-22\bin\java.exe" "-javaagent:C:\Users
Basic HTTP Server
[✓] Socket ready on port 5500
[✓] Open this link in the browser: http://localhost:5500

[✓] Client connected on /127.0.0.1:50524
    | GET / HTTP/1.1
    | Host: localhost
    | [✗] Connection closed on /127.0.0.1:50524
```

At the bottom of the console, a status message reads: `// Build completed successfully in 261 ms (2 minutes ago)`.

Figure 2.4: Updated output of the server side script.

Additionally, we can also type ***http://localhost:5500*** in a web browser and view the web page.

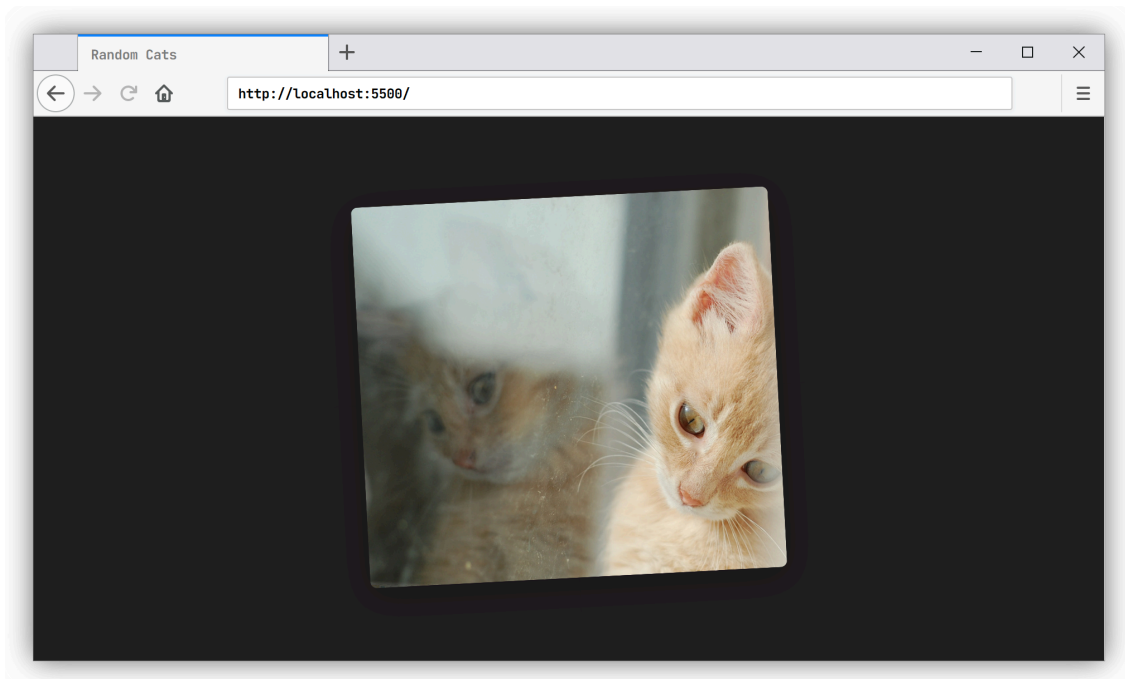


Figure 2.5: Viewing the server response using a web browser (Firefox).

When we send a client request using a web browser like Firefox, we get to see additional information about the connection in the server's output terminal:

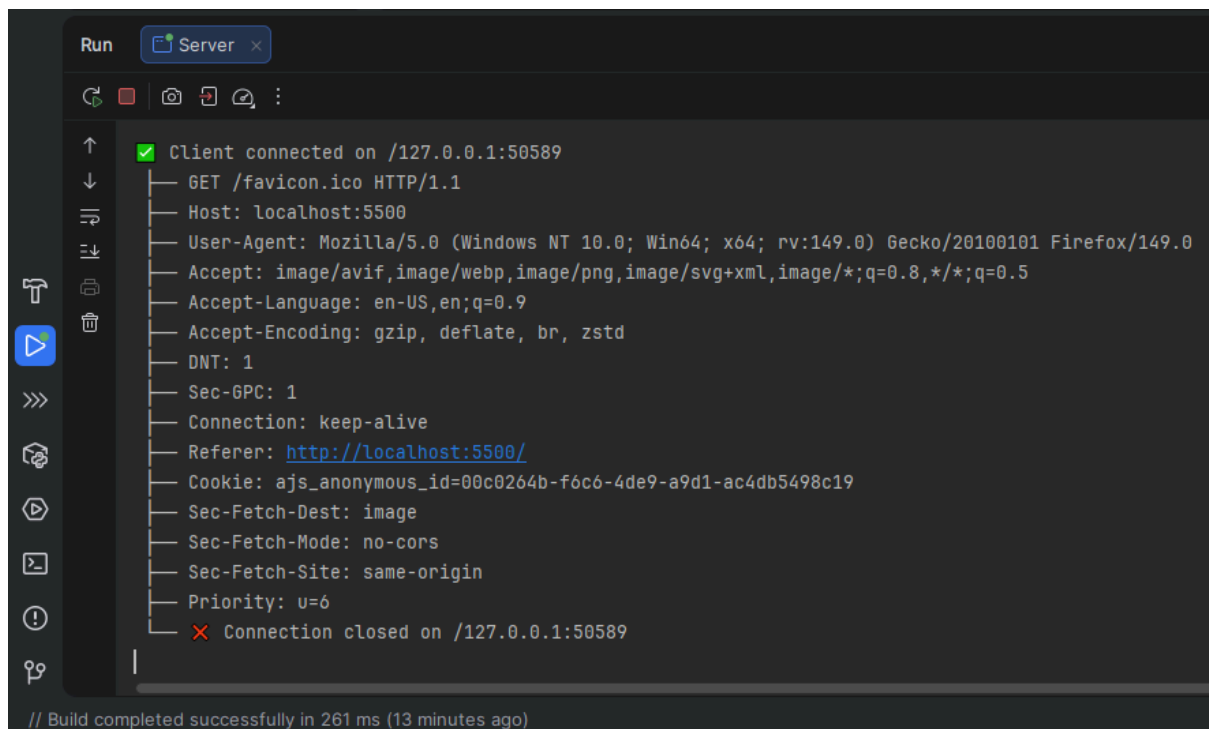


Figure 2.6: Additional information about the connection in the server output.

Conclusion

In this experiment, we have successfully implemented a basic HTTP client-server configuration using sockets where a client and a server can communicate with each other over the localhost. We examined the output, tested it using a web browser, and verified our implementation.

This experiment helped us gain foundational knowledge about HTTP protocol, request-response structure, and client-server architecture as well as practical skills to build a working web server from scratch using Java programming language.
